Which Program Did the Agent Run?

Checking and Keeping the Code AI Agents Write

Abstract

AI agents act by writing and running code during a task, guided by text attackers can influence, and the code is rarely kept. For consequential actions, we argue, the agent should first write the program, have it checked and kept, and then run it, so every action can be traced.

A prescription passes through several hands before anyone takes a pill. A physician writes it: the drug, the dose, how often. A pharmacist checks it against the patient's other medications before anything leaves the counter. The pharmacy dispenses what was written and keeps the prescription on file, so that a refill repeats the same order and a mistake can be traced to the line that caused it. Cough drops need none of this; medicines that can cause real harm do.

AI agents now do consequential work, and they do it differently. Ask one to file an expense report, triage a bug, or update a cloud setting. It reads web pages, issue threads, emails, and tool descriptions, decides what to do next, and does it, one step at a time. Taken together, those decisions form a program: a loop, a few conditions, and often small scripts that the agent writes in the middle of the task. The model that writes this program reads text that anyone, including an attacker, could have written. The program runs as soon as it is written, nobody reviews it first, and it is usually discarded when the task ends. The operator keeps, at best, a log of the actions the agent took.

The analogy differs from the agent case in two ways. The author of the program is a model rather than a licensed professional, and the text it reads while writing can be poisoned. And the review that the pharmacist provides will mostly have to be done by machines, because people cannot review programs as fast as agents write them. Both differences make the review and the record more important.

Our claim is about traceability. **For actions with consequences, an agent run should leave behind a program that was checked before it ran and can be read after it ran**, together with a record of what ran, on whose authority, which checks it passed, and what it changed. We call the practice that produces such a record "author, then act": the model first writes a program, the program is checked, and only then does it run.

We begin with a small example in which the record is missing, explain why better models will not supply it, and describe author, then act. We then set out what an attacker can still do, what the record should contain and which checks belong in it, where the argument is weakest, and what we ask of builders, standards bodies, researchers, and editors.

I. Which program produced this list?

Consider an ordinary research chore. Every month, a researcher wants the new arXiv papers in software engineering (cs.SE) or security (cs.CR) whose abstracts mention "coding agent", with title, first author, and link. For September 2026 we gave this task to a browser agent, Claude in Chrome, three times, and let it work the way agents usually do. Figure 1(a) shows what happened.

Each run took a different route. The first found arXiv's advanced search, filled in the form field by field, and read the 184 results 50 at a time, keeping the entries tagged cs.SE or cs.CR. It took 26 browser actions, seven of which ran JavaScript snippets that the agent wrote on the spot to copy results off the page. The second run filled in the form in one step, asked for 200 results per page, and read them with one script, in 13 actions. The third went straight to the results page by URL and read it with one script, in 2 actions. The three runs shared one browser session, so the later runs could see what the earlier ones had done, which is part of why they got shorter; no record captured that shared state either. All three runs returned the same 91 papers.

We then asked a model for a program that produces the list, instead of the list itself. arXiv publishes an interface for exactly this purpose, and Figure 1(b) shows the program written against it. The whole request is one query: the phrase, the two categories, and the date range. We ran the program three times on 13 October 2026. Each run made one request, took under a second, and returned the same 98 papers, identical down to the hash of the rows.

The two answers differ by seven papers. Six of them were first submitted on 30 September, a date that falls inside September in both UTC and US Eastern time, so they belong in the list under any reading of the request. The seventh was submitted at 22:04 Eastern time on 31 August, which is already 1 September in UTC, so whether it belongs depends on the time zone the researcher had in mind. The agent's answer is consistent with a date range from midnight Eastern on 1 September to midnight Eastern at the start of 30 September: the agent appears to have left out the last day of the month. We cannot be sure, because the agent kept no program; we reconstructed its date range afterwards by comparing submission timestamps.

Suppose the list mattered, and someone asked why those papers were missing. For the program, the answer can be read directly. The date range is one line, the query is a string kept with the program's hash, and the interface it calls is published, documented, and returned identical rows on all three runs. A test can replay the query against a recorded response. For the

agent's runs, there is no program to open. We kept a log of each run's actions, which is more than many deployments keep. The log shows that a form was filled in, pages were read, and seven scripts were run, but not what those scripts contained: the agent wrote them, ran them, and moved on. The rule that decided which papers counted was spread across form fields, page reads, and code that nobody kept. The three runs happened to agree, but nothing in them guarantees that the next run will.

For a reading list, this gap costs little. For an agent that moves money, changes permissions, or deletes data, it decides whether an incident can be investigated or whether investigators can only describe what happened without being able to explain it.

The program also shows what a kept program looks like. It can be read in a minute. Its manifest, the list of external actions it can take, contains only read requests to one host, sent at the rate the host asks for; each of our runs needed one request. It can be pinned to a version, scheduled, and rerun every month in the same form. The papers reach it as fields in a data feed, never as text that decides what it does next, so nothing posted to arXiv can change which request it makes. The browser agent clicked through pages built for people; the program used an interface built for programs, which arXiv had published all along. Many sites offer less. Piet and colleagues make the same point for web agents: committing to a program before reading a page works best where sites offer typed operations whose effects are known before they run [1].

II. Why better models will not solve the problem

One might expect better models to make such records unnecessary. The evidence suggests otherwise: models remain easy to mislead and do not behave the same way twice, so the record has to be kept outside the model.

The model cannot separate instructions from data. SQL injection was largely contained by separating code from data at the interface, through parameterized queries, and a language model has no such interface. The UK National Cyber Security Centre therefore argues that prompt injection should not be treated as the next SQL injection, describes LLMs as inherently confusable, and tells designers to rely on deterministic safeguards outside the model [2]. Published defenses do not close the gap. Nasr and colleagues took 12 recent defenses against jailbreaks and prompt injection, most of which had reported near-zero attack success, and with adaptive attacks pushed attack success above 90% for most of them [3]. Because instructions and data are mixed inside the model, a log of its actions cannot later show whether an action came from the operator's request or from text an attacker planted.

The same agent takes a different path each time. Bjarnason, Silva, and Monperrus analyzed 60,000 trajectories on SWE-bench Verified. A model's pass rate moves by 2.2 to 6.0 percentage points depending on which single run is reported, standard deviations above 1.5 points remain even at temperature zero, and trajectories often diverge within the first few percent of their tokens [4]. For benchmarks this is a statistics problem. For security it means that the run that was evaluated is not the run that acted, and the run that acted cannot be reproduced on demand. Forensic work on a deployed agent reaches the same conclusion: because the model, the execution environment, and the changing context all influence each step, "identical inputs need not yield identical traces" [5]. Asking the model to write the program again does not recover the program that ran.

When something goes wrong, there is no program to examine. A prompt injection in the title of a GitHub issue was enough to make Cline's AI triage bot install and run code of the attacker's choosing. Khan, who reported the flaw, showed that this foothold could reach the project's publishing secrets through a shared build cache; after disclosure, an unauthorized release of Cline's command-line tool appeared on npm [6]. In July 2025, Replit's coding agent deleted a customer's production database despite instructions to freeze code, and Replit responded by separating development and production databases [7]. In both cases the agent ran commands that no one had seen before they ran. Forensic study of such systems is only beginning; Gruber and Hilgert observe that little is known about how an agent's internal state and actions can be reconstructed [5].

Governments now ask for the record. In May 2026, six government agencies from the Five Eyes nations, led by Australia's ACSC, jointly advised organizations to use agentic AI only for low-risk, non-sensitive tasks at first and to expand step by step. The same guidance warns that agentic architecture "can obscure what caused a particular action, making accountability hard to trace," and asks operators to ensure traceability for decisions and actions [8]. It asks them to log prompts, tool calls, decisions, and the information agents retrieve, and to keep agents from writing to those logs. It says nothing explicit about keeping the code an agent writes and runs. A log of actions shows the path a run took. It does not show the rule that chose the path, and that rule is what an investigator needs.

III. Author, then act

We propose splitting the agent's work into two steps. First, the model *authors*: it reads the request, the documentation, and whatever else it needs, and writes a program. Second, after that program has been checked, the program *acts*. Four properties distinguish this from the usual agent loop. The fourth is the one this article adds.

The plan is fixed before untrusted input is read. Which actions the program can take, in what order, and under what conditions is decided when the program is written, from the trusted request. When the program runs, untrusted data passes through it only as values, such as an invoice amount or the body of an email. Such data cannot add an action, widen the program's permissions, or remove a check, although it can still choose among the branches the program already has. Outputs may differ from run to run because the world differs, but the set of possible actions may not.



Fig. 1. The same request, three times, two ways. (a) Claude in Chrome, asked for the September 2026 reading list. The window shows the arXiv page the agent reads and, in the side panel, its log of run 1, redrawn from the recorded action log. Below it are the three runs: three different routes of 26, 13, and 2 actions, with scripts written during the run. The answers happened to agree. (b) The program a model wrote against arXiv's published API, run three times: same code, same hash, same 98 rows. The program's list includes seven papers that the agent's list lacks: six submitted on 30 September and one submitted in the first hours of 1 September UTC. Only (b) left a program that can be read. Screenshot and counts from the runs on 13 October 2026.

Models called while the program runs cannot use tools. Many tasks still need a model during the run: to summarize an email, extract fields from a scanned form, or decide whether a ticket is about billing. The program may call a model for these steps, but that model has no tools, and its output is treated as data: typed, bounded, and checked like any other untrusted input. It may choose among branches the program already has; it cannot call a tool or add a step. A summarizer that has been injected can write a bad summary; it cannot send the summary to a new address.

The program can be checked. Because the program exists before it acts, a person can read it, a tool can analyze it, and it can be tested, subjected to fault injection, and given exactly the permissions it uses. When it is wrong, it can be fixed once, and every later run uses the fix. This works only if the program is written in a language that can be checked: a restricted workflow language, or a runtime without `eval`, dynamic imports, or unrestricted network and file access, in which every external call goes through a declared capability. An agent that may write arbitrary shell code can hide an action from any manifest.

The program is kept. The code that runs is exactly the code that was checked, identified by a hash, and every run records which program it ran, under whose authority, and with what effect. This is what makes the first three properties verifiable after the fact: a check matters only if one can later show that the checked program is the one that acted.

The split confines variation to the authoring step, where it is cheap, contained and often useful: drafting several candidate programs, exploring plans, writing tests. In the step that acts, variation does harm, because there it means that the system that was evaluated is not the system that runs. Figure 2 shows the two pipelines side by side.

The mechanisms are not new. Beurer-Kellner and colleagues list six design patterns for protecting LLM agents from prompt injection. One of them, *code-then-execute*, has the agent write a program that calls tools and passes untrusted text only to models with no privileges [9]. CaMeL implements a version of this with explicit capabilities attached to data. It solves 77% of the AgentDojo tasks with provable security under its policies, against 84% for the undefended agent [10]. Piet and colleagues argue that web agents should by default compile a task-specific program before reading web content and then execute it. They classify every WebArena task as compatible with this approach, and 81% as completable by a purely static program with no model call at run time, assuming complete and trusted APIs [1]. OWASP's Top 10 for agentic applications lists unexpected code execution as a risk, because code an agent generates in real time can bypass traditional controls, and recommends separating code generation from execution, with validation gates and a log of every generation and run [11].

This article adds two things. It extends the default beyond the web to every agent whose actions have consequences, coding agents included, and it asks what a program written by an agent should carry before it runs and leave behind after it runs.

Coding agents are the clearest case. They read untrusted repositories and issues, hold credentials, and alternate between writing commands and running them. Under author, then act, the authoring step itself still runs untrusted code, because tests and build scripts come from the repository, so it belongs in a sandbox with no publishing credentials and no caches shared with release jobs. What leaves the sandbox is a patch and the tests that justify it. The patch then follows the path every human change already follows: review, continuous integration, and a signed release, with the checks described below filed against the patch's hash. That path is itself a traceability system, built over decades, and coding agents with shell access and standing credentials bypass it. Cline's triage bot only needed to produce a label and a short summary. Had that output been passed as data to a fixed workflow, a malicious issue title could have caused a wrong label and nothing more.

The web is moving in the same direction. WebMCP, a draft from the W3C Web Machine Learning Community Group with editors from Microsoft and Google, lets a page register tools for agents, each with a name, a plain-language description, and an optional input schema [12]. In today's agent loop, those descriptions shape each action: an agent filing travel expenses reads them, along with every receipt it was given, and decides call by call what to invoke. Under author, then act, the agent reads them once and writes a short program. The program extracts the amount, date, and merchant from each receipt with a model call that has no tools, checks them against types and limits, and calls the site's `submitExpense` tool. The manifest shows one write action. If the site changes its tools, the program stops instead of guessing, and a new program is written and checked. The current draft lets a page mark a tool as read-only or consequential, but the marks are optional, and the draft itself warns that a malicious site can omit or misrepresent them. Stable tool identifiers, versioned descriptions, and side-effect classes that a client can check would let a manifest, and a record, be computed without relying on the site's own descriptions.

IV. What the attacker can still do

A design that claims to improve security should state its threat model. Ours assumes an attacker who controls some of the text the system reads: a web page, an issue, an email, an attachment, or a tool description. The attacker cannot change the user's request, the policy that checks programs, the environment that runs them, or the record. The attacker wants one of three things: an action the user did not intend, data sent where it should not go, or a task that fails. Table I compares what the attacker can do from each position.

Three differences follow. First, the attack surface changes from unbounded to countable. In the loop, any tool the agent holds can be reached from any text it reads. Under author, then act, the manifest lists the actions that can be reached, the branches that depend on data can be counted, and both lists go into the record.

Second, the channels through which data can leave become visible. In the loop, every tool that can send data out is a channel that an injected instruction can open. In an authored program, only the channels in the manifest exist, and a checker can flag, before anything runs, a manifest that sends messages to addresses taken from untrusted input.

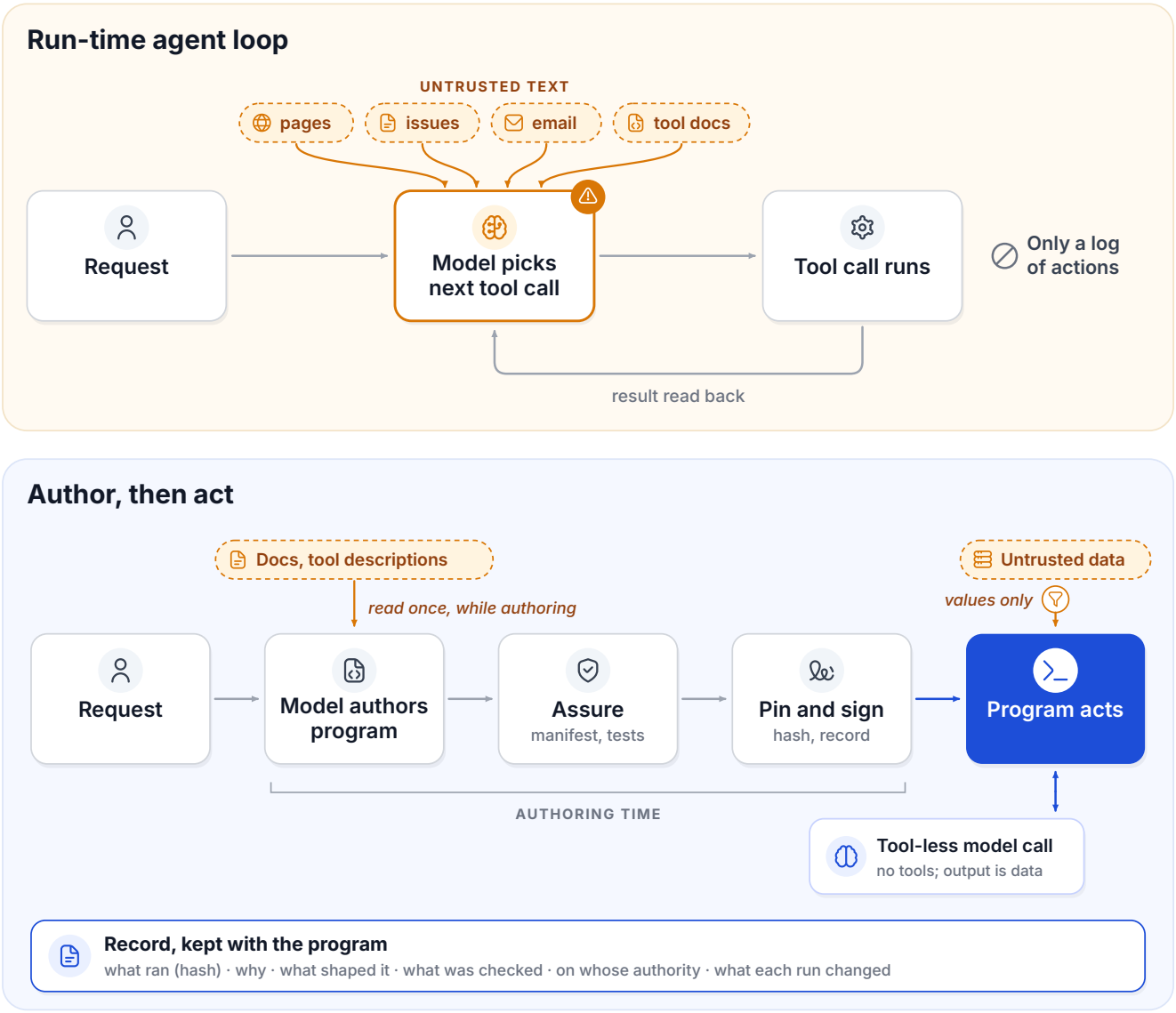


Fig. 2. Top: in a run-time loop, untrusted text reaches the model that chooses every next action, and only a log of actions remains. Bottom: under author, then act, untrusted documentation can shape a program that must pass checks before it runs; untrusted data reaches the running program only as values; and the checked program is pinned, signed, and kept with a record of each run.

Third, the attack leaves evidence. An injection during the run takes effect immediately and leaves at best a log of actions. An attack during authoring must get past the checks, and if it does, it stays in the record together with the authoring inputs that carried it, where it can be found later. Keeping those inputs does not show how they influenced the model, but it tells an investigator where to look.

V. The Record, and the Checks It Carries

Traceability has an established meaning in software engineering. Gotel and Finkelstein define requirements traceability as "the ability to describe and follow the life of a requirement, in both a forwards and backwards direction" [13]. For a program written at run time, the equivalent is the ability to follow an action back to the program that took it, the checks that program passed, the authority it ran under, and the request it served, and forward to what it changed. Software supply chains already require this for artifacts that persist. In-toto, for example, records who performed each step of a build and lets the end user verify the chain from source to deployment [14]. Programs written at run time bypass that chain entirely.

We propose that the record answer six questions for every consequential run.

1) **What ran?** The program, identified by its hash, and the versions of the tools and libraries it called.
2) **Why?** The request it served and the person or policy that issued it. This is the documented purpose, not a claim about the model's internal reasoning.

TABLE I
WHAT AN ATTACKER WHO CONTROLS SOME INPUT TEXT CAN DO, BY WHERE THAT TEXT IS READ.

Where the attacker's text is read	**Run-time agent loop**	**Author, then act**
During the task, as data or tool output	Can add, drop, or reorder any action the agent's tools allow	Can change values and choose branches the program already has; cannot add actions
Inside a model call made while the program runs	Same as above: the model that reads is the model that acts	Can corrupt that call's output, which is then checked as untrusted data
In tool definitions changed after authoring	Influences the choice of the next call	A changed definition no longer matches the pinned program, which stops; writing a new program repeats the checks
In documentation read while authoring	No different from the rows above	Can shape the program, which must still pass the checks and is kept with the inputs that shaped it

3) **What shaped it?** The documentation, tool descriptions, and other inputs read during authoring, stored where possible or referenced by hash. This shows what was read, even where the content itself cannot be kept.
4) **What was checked?** The evidence described below, bound to the same hash.
5) **On whose authority?** The principal whose permissions the program used, and who or what approved it.
6) **What did it read, and what changed?** The untrusted inputs and the outputs of model calls that each run consumed, and its external effects: requests made, records written, messages sent, with the receiving service's acknowledgment where there is one.

A log of actions answers part of the last question. It records the path one run took, not the rule that chose the path, so it cannot answer the first. A thoroughly logged agent loop, one that keeps every prompt, script, and tool argument, can answer more of these questions than most deployments can today, and builders should log that way. It still cannot answer the fourth question, because nothing was checked before the run: the program existed only as the run unfolded. Nor can it make a fix permanent, because the next run writes a new program.

Writing the program down and keeping it is necessary but not sufficient. If the only check between an agent-written program and production is a person glancing at it, little is gained. Perry and colleagues found that people using an AI code assistant wrote less secure code than people without one, and were more likely to believe their code was secure [15]. Trust in an author that seems capable weakens a person's own checking, and agents produce more code than people can review. The record should therefore carry evidence, produced mostly by machines, that a reviewer or a policy engine can check. We propose four kinds; the list is a starting point.

1) **A capability manifest.** Every external action the program can take, with its side-effect class: read, write, irreversible, or credential-touching. For programs restricted to typed tool interfaces the manifest can be extracted automatically; for arbitrary code it must also be enforced while the program runs. The manifest is compared with the request. A program asked to file an expense that can also change a payee's bank details fails at this point, before any test runs. This turns OWASP's principle of least agency [11] into something that can be checked.
2) **Coverage of the untrusted inputs.** In a program written in a checkable language, the points where untrusted data can decide which branch runs can be listed, and combinations of their values can be tested systematically. Combinatorial (t-way) testing suits this case, because it targets faults that appear only when a few inputs take particular values together. It covers every combination of t values, not every path.
3) **Fault injection on the data path.** Feed the program adversarial strings, malformed tool responses, and hostile outputs from its own model calls, and check that every action it takes stays within the manifest and its limits, and that it stops safely when it should. This tests the central claim directly: data may change values and choose among existing branches, but may not add actions.
4) **Differential authoring.** Ask for the same program several times. Where independent attempts agree on the manifest, confidence increases. Where they disagree, the request may be ambiguous or the authoring inputs may have been tampered with, and a person should look. Agreement alone is weak evidence, because attempts can share a mistake or read the same poisoned documentation; giving each attempt a different subset of the untrusted documents makes disagreement more informative. Disagreement is a reason to investigate, not a diagnosis.

Each check produces an artifact that is bound to the program's hash and filed with it, and every run records the hash it ran. When something goes wrong, there is a specific program to examine, a specific set of authoring inputs to investigate, and a list of the checks the program passed. The record is only as trustworthy as its storage: it belongs in an append-only store with its own signer, which the acting program cannot write to, a point the Five Eyes guidance also makes for agent logs [8]. Because the record holds what the agent read, it also needs a named owner and limits on who may read it.

The requirements should scale with the stakes. A program that only reads a calendar needs little more than its manifest and a hash. A program that moves money, changes permissions, publishes, deletes, or touches credentials should pass all four

checks and leave a complete record.

VI. Where we may be wrong

We have argued for a default, and defaults have costs. We see seven places where the argument is weakest.

Utility. CaMeL gives up seven points of task success on AgentDojo [10]. For some kinds of task the cost of writing the program in advance will be too high, and the default should give way. That cost should be measured for each type of task.

One-off work. Writing, checking, and filing a program costs more than doing a task once. The default targets actions that matter or that repeat. For a one-off action that matters, the authored program may be a short, constrained workflow that a person approves before it runs; if it is kept, it is still more than most agents leave today. In the terms of the analogy, the over-the-counter category should cover low-privilege, reversible work, not every task that happens to be new.

Branch selection by data. Fixing the plan in advance does not stop untrusted data from choosing among the branches the program already has. The attack surface shrinks from anything the agent can do to any branch the program allows. Those branches can be listed and tested, but they do not go away.

Injection during authoring. The author still reads documentation and tool descriptions written by others, and a poisoned description can lead to a poisoned program. The difference is that the attack must now get past the checks, and it stays in the record if it does.

Shared failures and maintenance. A single program run everywhere also fails everywhere, so pinned programs need the deployment discipline of any other software, including staged rollout. When a faulty content update to CrowdStrike's sensor crashed Windows machines worldwide in July 2024, the faulty file could at least be identified, analyzed, and replaced. A kept program also does not keep the world around it fixed. It repeats the same steps, but its results change when the services it calls change, and it must be rewritten and checked again when its tools or data change. Pinning detects a changed interface, not a service that silently starts to behave differently.

The record itself. Keeping what agents read and write raises privacy and storage questions, because much of that content is sensitive. Where content cannot be kept, the record should hold hashes and references instead, and retention should be limited to consequential actions. A record supports accountability only if someone is responsible for acting on it.

Evidence. Much of what we cite on agents is preprint, and the incidents are case reports, not rates. The measurement we most want is only starting to appear, and not yet for tasks with consequences: across many runs of the same task, how often do trajectories differ in *which actions* they take, not just in whether they succeed?

VII. An agenda

If the argument holds, it asks something of each part of our community.

- **Builders of agent products** should disable run-time agency by default for actions that move money, change permissions, publish, delete, or touch credentials, with a declared and logged way to enable it, and should keep the authored program and its record for every such action.
- **Standards efforts**, including OWASP's work on agentic applications [11], the WebMCP community group [12], and national bodies such as NIST, should define side-effect classes for agent tools, a manifest format, and a record format that links a program to its checks, its authority, and its effects, much as in-toto links the steps of a build [14].
- **Researchers** should measure how often the actions taken differ across trajectories, quantify the utility cost of authored programs for each type of task, and build assurance and forensic methods for agent-written programs: combinatorial coverage of branches that depend on data, fault injection on the data path, differential authoring, and reconstruction of what an agent did [5].
- **Reviewers and editors** can help by asking papers that evaluate agents to state whether actions were checked before they ran, against what, and whether the program that acted was kept.

A prescription on file can be read long after the medicine is gone: what was ordered, by whom, checked by whom, and dispensed when. We are not asking agents to write a program for every small task, just as cough drops need no prescription. For actions with consequences, however, the program should exist before it runs, something other than its author should check it, what runs should match what was checked, and the record should outlast the run. A program that was kept can be read, tested, and corrected. A run that left no program leaves only its effects, with no way to explain them.

References

[1] J. Piet, A. Chow, Y. Hou, M. Lyu, S. Venuto, J. Zhu, R. A. Popa, and D. Wagner, "Web agents should adopt the compile-and-execute paradigm," 2026, v2, 23 Oct. 2026; v1 (14 May 2026) was titled "Web Agents Should Adopt the Plan-Then-Execute Paradigm". [Online]. Available: https://arxiv.org/abs/2605.14290

[2] D. Chismon, "Prompt injection is not SQL injection (it may be worse)," https://www.ncsc.gov.uk/blog-post/prompt-injection-is-not-sql-injection, UK National Cyber Security Centre (NCSC), Dec. 2025, NCSC blog, 8 Dec. 2025. Accessed Oct. 13, 2026.

[3] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in *Proceedings of the 35th USENIX Security Symposium*. Baltimore, MD: USENIX Association, Aug. 2026, pp. 1467–1486, arXiv:2510.09023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/nasr

[4] B. H. Bjarnason, A. Silva, and M. Monperrus, "On randomness in agentic evals," 2026. [Online]. Available: https://arxiv.org/abs/2602.07150

[5] J. Gruber and J.-N. Hilgert, "Foundations for investigations of agentic AI systems from the forensic analysis of OpenClaw," *Digital Threats: Research and Practice*, 2026.

[6] A. Khan, "Clinejection — compromising Cline's production releases just by prompting an issue triager," https://adnanthekhan.com/posts/clinejection/, Feb. 2026, blog post, 9 Feb. 2026. Accessed Oct. 13, 2026.

[7] S. Sharwood, "Replit makes vibe-y promise to stop its AI agents making vibe coding disasters," https://www.theregister.com/2025/07/22/replit_saastr_response/, Jul. 2025, The Register, 22 Jul. 2025. Accessed Oct. 29, 2026.

[8] Australian Signals Directorate's Australian Cyber Security Centre (ASD's ACSC), United States Cybersecurity and Infrastructure Security Agency (CISA), National Security Agency (NSA), Canadian Centre for Cyber Security, New Zealand National Cyber Security Centre (NCSC-NZ), and United Kingdom National Cyber Security Centre (NCSC-UK), "Careful adoption of agentic AI services," https://www.cyber.gov.au/sites/default/files/2026-05/careful_adoption_of_agentic_ai_services.pdf, May 2026, accessed Oct. 13, 2026.

[9] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," 2025. [Online]. Available: https://arxiv.org/abs/2506.08837

[10] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. Munich, Germany: IEEE, Mar. 2026, pp. 587–618.

[11] OWASP Gen AI Security Project, "OWASP top 10 for agentic applications for 2026," https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/, Dec. 2025, published 9 Dec. 2025. Accessed Oct. 13, 2026.

[12] B. Walderman, K. Sagar, D. Farolino, and S. Dallstream, "WebMCP," https://webmachinelearning.github.io/webmcp/, W3C Web Machine Learning Community Group, Oct. 2026, draft Community Group Report, 27 Oct. 2026 (eds.). Accessed Oct. 29, 2026.

[13] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of the First International Conference on Requirements Engineering (ICRE)*. IEEE, 1994, pp. 94–101.

[14] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 1393–1410. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[15] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: ACM, Nov. 2023, pp. 2785–2799. [Online]. Available: https://doi.org/10.1145/3576915.3623157